In this tutorial we focus on three classical algorithms in graph theory. Each of them solves a fundamental problem related to shortest paths or minimum spanning trees (MSTs). We highlight the basic idea behind each algorithm and analyze their running times.

- Dijkstra: shortest paths in graphs with non-negative weights.
- Prim: MST by growing a tree from a vertex.
- Kruskal: MST by adding edges in sorted order.

Dijkstra's Algorithm (Shortest Paths)

Dijkstra's algorithm computes the shortest path from a single source vertex s to every other vertex in a weighted graph, assuming all edge weights are non-negative. The algorithm maintains a set of vertices for which the shortest distance is already known and repeatedly selects the next vertex with the smallest tentative distance, relaxing the edges incident to it.

Time Complexity:

- With a min heap (priority queue): $O(m \log n)$.

Prim's Algorithm (Minimum Spanning Tree)

Prim's algorithm grows an MST starting from any vertex. At each step it adds the lightest edge that connects a vertex inside the growing tree to a vertex outside. This greedy choice ensures that the resulting spanning tree is minimum weight.

Time Complexity:

- With a min heap (priority queue): $O(m \log n)$.

Kruskal's Algorithm (Minimum Spanning Tree)

Kruskal's algorithm constructs an MST by considering edges in order of increasing weight. An edge is added if it does not form a cycle with the edges already chosen. The algorithm stops once $n - 1$ edges are selected.

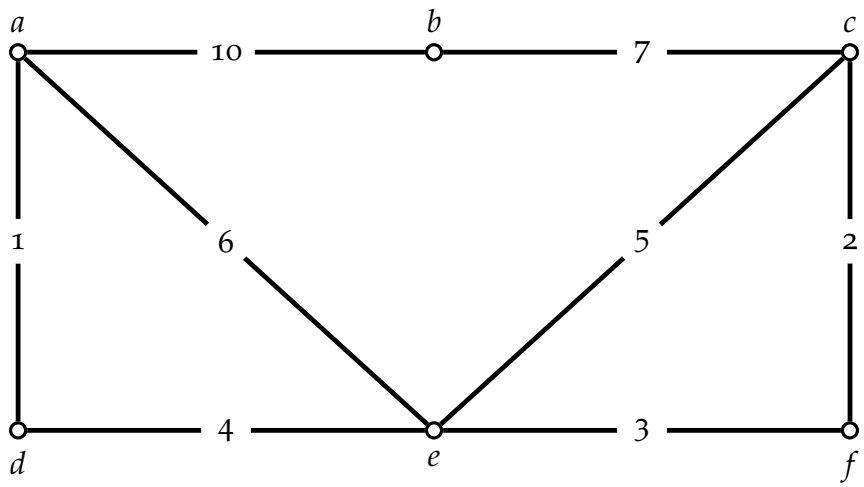
Time Complexity:

- With cycle detection using DFS: $O(mn)$. (Each connectivity check can take $O(n)$ time in the current forest, and up to m edges may be checked.)
- With Union–Find: $O(m \log m + n^2)$. Here, sorting edges costs $O(m \log m)$ and each find is $O(1)$ and each union is $O(n)$ in naive union–find (Without union by rank + path compression) where union–find operations take $O(n^2)$ total across the run.

Warm-up

Problem 1. Consider Dijkstra's shortest path algorithm for undirected graphs. What changes (if any) do we need to make to this algorithm for it to work for directed graphs and maintain its running time?

Problem 2. Consider the following weighted undirected graph G:



Your task is to compute a minimum weight spanning tree T of G:

a) Which edges are part of the MST?

b) In which order does Kruskal's algorithm add these edges to the solution.

c) In which order does Prim's algorithm (starting from a) add these edges to the solution.

Problem solving

Problem 3. Let $G = (V, E)$ be an undirected graph with strictly positive edge weights $w : E \to \mathbb{R}^+$. For each edge $e \in E$ define three transformed weight functions:

$$w_1(e) = \alpha\, w(e) \quad (\alpha > 0), \qquad w_2(e) = w(e) + \beta \quad (\beta > 0), \qquad w_3(e) = \big(w(e)\big)^2.$$

Throughout, all weights remain positive. Consider the effect of these transformations on shortest paths and on minimum spanning trees.

a) Let p be a shortest s–t path under w. Is p still optimal under w_1? under w_2? under w_3? Give brief justifications or counterexamples.

b) Let T be a minimum spanning tree under w. Is T still optimal under w_1? under w_2? under w_3? Explain your reasoning.

Problem 4. In many optimization problems there can be more than one optimal solution. For instance, in the shortest s–t path problem there might be several different paths from s to t with the same minimum length. In such cases we may want to break ties by preferring the path that uses the fewest edges.

Show how to reduce this to an ordinary shortest path problem by modifying edge lengths. Assume all original edge lengths $\ell(e)$ are positive integers.

a) Define new edge lengths $\ell'(e) = M \cdot \ell(e)$ for each edge e. Prove that if P and Q are two s–t paths with $\ell(P) < \ell(Q)$, then $\ell'(Q) - \ell'(P) \geq M$.

b) Define new edge lengths $\ell''(e) = M \cdot \ell(e) + 1$ for each edge e. Prove that if P and Q are two s–t paths with $\ell(P) = \ell(Q)$ but P uses fewer edges than Q, then $\ell''(P) < \ell''(Q)$.

c) Show how to choose M so that the path that is shortest under ℓ'' is guaranteed to also be shortest under ℓ and, among all such shortest paths, uses the fewest edges.

Problem 5. You are given a graph where each edge e has a length $\ell(e) > 0$ and each vertex v has a cost $c(v) > 0$. For two specified vertices s and t, the cost of a path $P = (v_0 = s, v_1, \ldots, v_k = t)$ is

$$\text{cost}(P) \;=\; \sum_{i=0}^{k} c(v_i) \;+\; \sum_{i=0}^{k-1} \ell(v_i, v_{i+1}).$$

Design an efficient algorithm that finds an s–t path of minimum cost.

Problem 6. Consider the IMPROVING-MST algorithm for the MST problem.

```
function IMPROVING-MST(G, w)
    T ← some spanning tree of G
    for e ∈ E \ T do
        T ← T + e
        C ← unique cycle in T
        f ← heaviest edge in C
        T ← T − f
    return T
```

Prove its correctness and analyze its time complexity. To simplify things, you may assume all edge weights are distinct.

Problem 7. Consider the REVERSE-MST algorithm for the MST problem.

```
function REVERSE-MST(G, w)
    Sort edges in decreasing weight w
    T ← E
    for e ∈ E in decreasing weight do
        if T − e is connected then
            T ← T − e
    return T
```

Prove its correctness and analyze its time complexity. To simplify things, you can assume the edge weights are distinct.

Problem 8. A computer network is modeled as an undirected graph $G = (V, E)$ where vertices are computers and each edge $e \in E$ has a *bandwidth* $b(e) > 0$. The bandwidth of a path P is the minimum edge bandwidth along P, i.e. $b(P) = \min_{e \in P} b(e)$ (also called the *bottleneck* of P).

Number the vertices $V = \{v_1, \ldots, v_n\}$. Let $B \in \mathbb{R}^{n \times n}$ be the matrix where B_{ij} is the maximum possible bandwidth between v_i and v_j over all paths from v_i to v_j (take $B_{ij} = 0$ if no path exists). Give an $O(n^3)$-time algorithm to compute B.

Challenge

This question is provided primarily to explore the boundaries of your problem-solving ability. It is intended as a stretch exercise and does not reflect the standard level of difficulty expected in the course.

Problem 9. A country has n cities, each of which needs access to a library. The government can either:

- Build a library in a city at cost c_{lib}, or
- Build a road between two cities *from the given candidate roads* at cost c_{road}. If there is a path of roads from city u to city v, then both cities can share a single library.

The goal is to decide:

- In which cities libraries should be built,
- Which roads should be constructed,
- Such that every city has access to some library and the total cost is minimized.

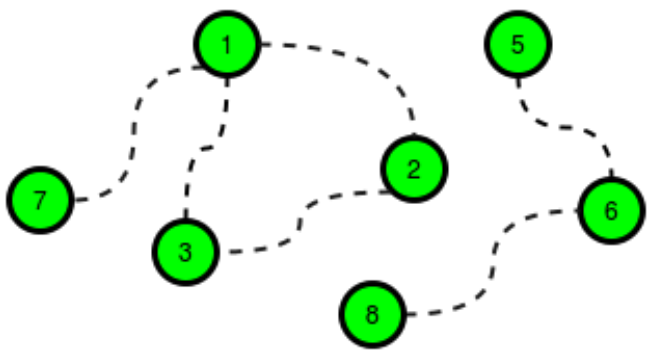


Figure 1: Sample map of cities and candidate roads

You may build roads only between pairs listed in the provided candidate set (as shown in the map). Both costs are uniform: every library costs c_{lib}, and every road costs c_{road}, independent of location.

a) Model this situation as a graph problem: what do vertices and edges represent?

b) Suppose $c_{road} \geq c_{lib}$. Where should libraries be placed, and are roads needed?

c) Suppose $c_{road} < c_{lib}$. Explain how you would decide both *where to build libraries* and *which roads to construct*.